

Lab: Intro to Python 1

Hakyung Sung
University of Hawaii at Manoa

August 26, 2026

Topics to cover

- Installation

Topics to cover

- Installation
- Environment management

Topics to cover

- Installation
- Environment management
- Basics I

Installation

Installation

- Python is a widely used, general-purpose programming language that is easy to learn, read, and write.

Installation

- Python is a widely used, general-purpose programming language that is easy to learn, read, and write.
- Vibe coding is about telling an AI what you want the code to do and letting it help write and revise the code.

cf. Google colab

- Browser-based notebook environment
- Comes with many common Python libraries
- Works well for quick experiments and demonstrations
- Vibe coding demo

Visual Studio Code

- VS Code
- Lightweight but powerful code editor
- Supports Python through extensions
- Includes terminal, linting, debugging, and version control tools
- Install both python and vsCode

Visual Studio Code: Chat

- Create a GitHub account if you do not already have one.
- In VS Code, click the Copilot/Chat icon and sign in with your GitHub account.
- No separate Chat extension installation is required.
- Open the Chat panel, which you can ask questions about your code.
- See the VS Code Chat overview for more details.

GitHub Copilot Chat

- GitHub Copilot Chat has a free tier for users with a GitHub account.
- No credit card is required to start with Copilot Free.
- The free tier currently includes:
 - 2,000 code completions per month
 - 50 chat requests per month
- Enough for basic class activities and experimenting with vibe coding, but not unlimited.

More info: [GitHub Copilot Plans & Pricing](#)

Environment management

Environment management

- A Python *environment* is an isolated workspace for a project.
- Different projects may require different:
 - Python versions
 - package versions
- Separate environments help (1) prevent version conflicts and (2) replicate the project.
- Common tools:
 - `venv` – built into Python
 - `conda` – manages Python and other dependencies

Before creating an environment

First, create a folder for this course.

For example:

SLS775

Then open the folder in VS Code:

1. Open VS Code.
2. Go to File → Open Folder...
3. Select your SLS775 folder.

You can keep our Python files and virtual environment inside this folder.

Example: venv

Create an environment (Suppose 'myenv' is the name of the environment)

macOS / Linux

```
python3 -m venv myenv
```

Windows

```
python -m venv myenv
```

Example: venv

Activate it:

macOS / Linux

```
source myenv/bin/activate
```

Windows

```
myenv\Scripts\activate
```

Deactivate it:

```
deactivate
```


Basics I

Basics I: Learning how to communicate with code

- The goal is to understand the basic language of Python.
- You will learn what *values*, *variables*, *functions*, and *methods* mean.
- Think of these as the basic concepts you need to read, understand, and communicate with code.

Goal: When you see Python code, you should have some idea of what it is asking the computer to do.

Basics I.1. Values: Different kinds of data

```
text = "apple"  
number = 10  
decimal = 2.5  
is_ready = True  
empty = None
```

What does this code show?

- Python works with different kinds of **values**
- "apple" is text
- 10 is a whole number
- 2.5 is a decimal number
- True represents a true/false value
- None represents no value

Values are the basic pieces of data that Python works with.

Basics I.1. Values: Strings

```
greeting = "Hello, world!"  
name = 'Python'
```

```
print(greeting)  
print(name)
```

Try it in VS Code

- Create a new file and save it (e.g., `values1.py`).
- To run the entire file, click ► **Run Python File** at the top right.
- To run only selected line(s), highlight them and press **Shift + Enter**.
- Check the output in the **Terminal** at the bottom of VS Code.

What does this code do?

- Creates two pieces of text
- Text values are called **strings**
- Strings are surrounded by quotation marks
- Either double quotes or single quotes can be used
- `print()` shows the value on the screen

Basics I.1. Values: The type of a value matters

"9" + "3"

9 + 3

"9" + 3

What happens here?

- "9" and "3" are strings, not numbers
- Adding two strings joins them: "93"
- Adding two numbers gives: 12
- Python cannot directly add a string and a number
- The **type** of a value affects what Python can do with it

Basics I.1. Values: Integers and floats

```
count = 7  
price = 3.5
```

```
count + 1  
price * 2
```

```
7 / 2  
7 // 2
```

What does this code show?

- 7 is an **integer**: a whole number
- 3.5 is a **float**: a decimal number
- Numbers can be used in calculations
- / performs regular division
- // performs floor division and removes the decimal part

Basics I.1. Values: Booleans

`5 > 3`

`2 == 4`

`True` and `False`

What does this code show?

- A **Boolean** can have only two values: `True` or `False`
- Comparisons produce Boolean values
- `5 > 3` asks whether 5 is greater than 3
- `==` asks whether two values are equal
- Boolean values are often used when code needs to make decisions

Basics I.1. Values: None

```
result = None
```

```
print(result)
```

What does this code show?

- None means that there is currently **no value**
- It is different from 0
- It is also different from an empty string such as ""
- You will often see None when a result is missing or has not yet been created

Basics I.1. Values: Put them together

```
text = "apple"  
number = 10  
decimal = 2.5  
is_ready = True  
empty = None  
  
print(text)  
print(number)  
print(decimal)  
print(is_ready)  
print(empty)
```

What should you notice?

- Python can store many different kinds of values
- Here we have text, numbers, a Boolean, and None
- `print()` lets us inspect each value
- You do not need to memorize these yet
- For now, recognize that **different values behave differently**

Basics I.2. Variables: Use a variable

```
language = "Python"
```

```
print(language)
```

What does this code do?

- Stores "Python" under the name language
- Later, we can use the variable name instead of writing the value again
- `print(language)` asks Python to retrieve and display the stored value

language → "Python"

Basics I.2. Variables: Update a variable

```
message = "I am learning"  
  
message = message + " Python"  
  
print(message)
```

What happens here?

- First, message stores "I am learning"
- Python reads the current value of message
- It adds the string " Python"
- The new value is stored back in message
- The final value is: "I am learning Python"

Basics I.2. Variables: = does not mean “equals”

```
message = "Hello"  
  
message = "Goodbye"  
  
print(message)
```

What should you notice?

- In Python, = means **assign**
- It tells Python: “store this value under this name”
- A variable can be assigned a new value later
- Here, "Goodbye" replaces "Hello"

Think of = as “**gets the value**”, not mathematical equality.

Basics I.2. Variables: Practice

```
favorite_food = "pasta"  
times_per_week = 2
```

```
print(favorite_food)  
print(times_per_week)
```

Try it yourself

- Run the code and check the output
- Change "pasta" to another food
- Change 2 to another number
- Run the code again
- Notice how changing the stored values changes the output

Basics I.3. Functions: Ask Python to do something

```
word = "banana"

print(word)
print(len(word))
```

What does this code show?

- A **function** performs an operation
- The value inside (...) is given to the function
- `print(word)` displays the value
- `len(word)` calculates the length of the string
- `len("banana")` returns 6

Function: give Python something → ask it to do something

Basics I.3. Functions: Input and result

```
word = "banana"

length = len(word)

print(length)
```

What happens here?

- word stores the string "banana"
- len(word) takes word as its input
- It produces the result 6
- That result is stored in length
- We can then use the result later

"banana" → len() → 6

Basics I.3. Functions: Some common functions

```
print("Hi")
```

```
len("abc")
```

```
str(16)
```

```
int(3.9)
```

```
float("2.5")
```

What do these functions do?

- `print()` displays a value
- `len()` returns the length
- `str()` converts a value to a string
- `int()` converts a value to an integer
- `float()` converts a value to a float

Basics I.3. Functions: Converting values

```
str(123)
```

```
int(4.8)
```

```
float("7.2")
```

What happens here?

- Functions can change the **type** of a value
- `str(123)` produces the string "123"
- `int(4.8)` produces the integer 4
- `float("7.2")` produces the float 7.2
- This is called **type conversion**

The value may look similar, but its **type** can be different.

Basics I.3. Functions: Functions can be nested

```
word = "banana"  
  
print(len(word))
```

How does Python read this?

1. Read the value of `word`
2. Run `len(word)`
3. Get the result 6
4. Run `print(6)`

Python generally evaluates the **inside function first**.

Basics I.3. Functions: Put them together

```
word = "banana"

print(word)
print(len(word))
print(str(123))
print(int(4.8))
print(float("7.2"))
```

What should you notice?

- Functions are written with parentheses:
function(...)
- We put an input inside the parentheses
- Different functions perform different operations
- Some functions return a new value
- That result can be printed, stored, or used by another function

Basics I.3. Functions: Practice

```
sentence = "I like studying Python"
```

```
print(len(sentence))
```

```
print(str(42))
```

Try it yourself

- Run the code and inspect the output
- Change the sentence and run it again
- Predict the output of `len(sentence)` first
- Change 42 to another number
- Try storing a function result in a variable

Basics I.4. Methods: Functions attached to an object

```
sample = "This is a STRING"  
  
sample.lower()
```

What does this code show?

- A **method** is a function attached to an object
- Here, `sample` is a string object
- The dot `.` connects the object to a method
- `.lower()` changes the text to lowercase
- The result is `"this is a string"`

`object.method()` = ask an object to do something

Basics I.4. Methods: .lower()

```
sample = "This is a STRING"

lower_sample = sample.lower()

print(lower_sample)
```

What happens here?

- `sample.lower()` operates on the string stored in `sample`
- It produces a new lowercase string
- The result is stored in `lower_sample`
- The original value of `sample` is not changed

Basics I.4. Methods: `.split()`

```
sample = "This is a STRING"
```

```
sample.split()
```

What does this method do?

- `.split()` separates a string into pieces
- By default, it splits the text at spaces
- The result is: ["This", "is", "a", "STRING"]
- The result is a collection of separate values called a **list**

Basics I.4. Methods: Chain methods together

```
sample = "This is a STRING"
```

```
sample.lower().split()
```

How does Python read this?

1. Start with sample
2. Run `.lower()`
3. Get `"this is a string"`
4. Run `.split()` on that result
5. Get `["this", "is", "a", "string"]`

Each method works on the result produced by the previous step.

Basics I.4. Methods: Functions + methods

```
sample = "This is a STRING"
```

```
len(sample.lower().split())
```

How does Python read this?

1. `sample.lower()` converts the text to lowercase
2. `.split()` separates it into words
3. `len(...)` counts the resulting items
4. The final result is 4

`sample` → `.lower()` → `.split()` → `len()` → 4

Basics I.4. Methods: Function vs. method

```
len(sample)
```

```
sample.lower()
```

```
sample.split()
```

What is the difference?

- A **function** usually looks like: `function(object)`
- A **method** usually looks like: `object.method()`
- Both perform operations
- The important thing for now is to recognize the two patterns when reading code

Do not worry about the technical distinction yet—learn to recognize the pattern.

Basics I.4. Methods: Practice

```
my_text = "Python is Fun"

print(my_text.lower())
print(my_text.split())
print(len(my_text.split()))
```

Try it yourself

- Run each line and inspect the output
- Predict the result before running it
- Change the text and run the code again
- Try
`my_text.lower().split()`
- Notice how methods can be combined step by step

Basics I.5. Exercise 1

`Exercise1_Firstname_Lastname.py`

Complete the exercises from Basics I in this file.

Your task

- Create one Python file
- Complete the assigned exercises
- Run your code and check the output
- Save the file using the required filename

Filename: `Exercise1_Firstname_Lastname.py`

Basics I.5. Exercise 1: Using VS Code Chat

- You may use VS Code Chat while completing the exercise.
- Use it to:
 - ask what a piece of code means;
 - ask why you are getting an error;
 - ask how to modify your code;
 - check whether your code does what you intended.
- Try to use **one chat session** for this exercise.
- Your goal is not simply to get the answer from AI, but to use the conversation to **understand and work with the code**.

Ask questions, test the suggestion, and inspect what happens.

Basics I.5. Exercise 1: Save your chat record

- After completing the exercise, export your VS Code Chat session.
- Save the exported chat record as:

`Exercise1_Firstname_Lastname_chat.json`

- Submit both:
 - `Exercise1_Firstname_Lastname.py`
 - `Exercise1_Firstname_Lastname_chat.json`
- I will demonstrate in class how to export the chat record from VS Code.